

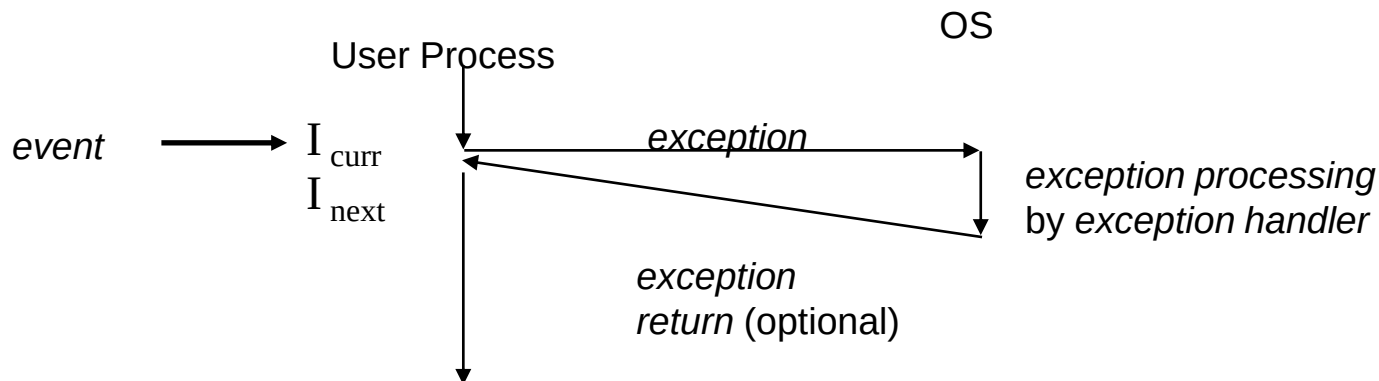


处理器

- 4.1 CPU概述
- 4.2 CPU设计的基本问题
- 4.3 MIPS模型机
- 4.4 单周期处理器设计
- 4.5 流水线技术
- 4.6 多周期处理器设计
- 4.7 流水线冒险
- 4.8 异常处理
- 4.9 指令间的并行
- 4.10 流水线结构实例

4.8 异常处理

- **异常**：除分支外，改变正常指令执行顺序的事件
 - 异常和中断处理程序工作在高特权级
 - 有些体系结构不区分异常和中断，统称为中断
 - MIPS系统
 - **异常** (Exception)：控制流中任何意外的改变，由当前正在执行的指令引发
 - 如系统调用，非法指令，页错误，溢出等
 - **中断** (Interrupt)：来自外部的事件，与当前执行指令无关



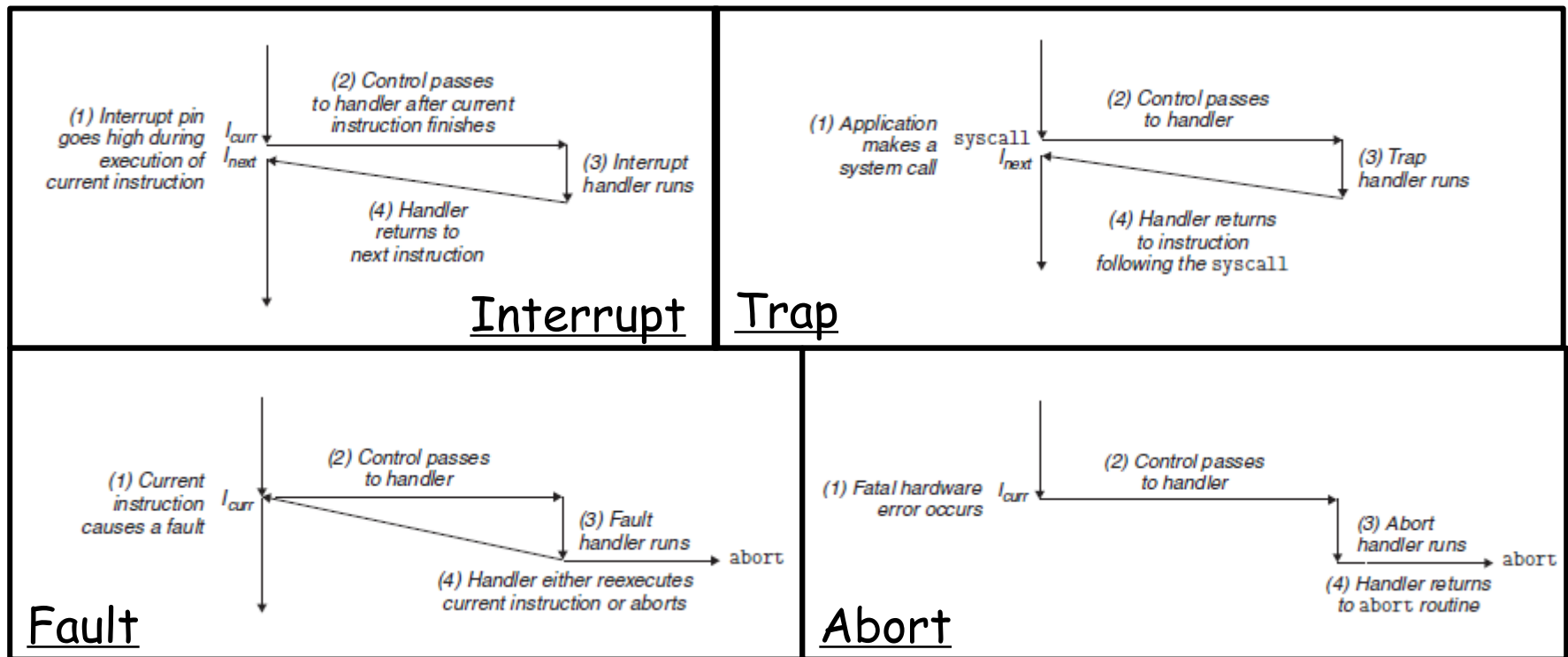


异常与中断分类

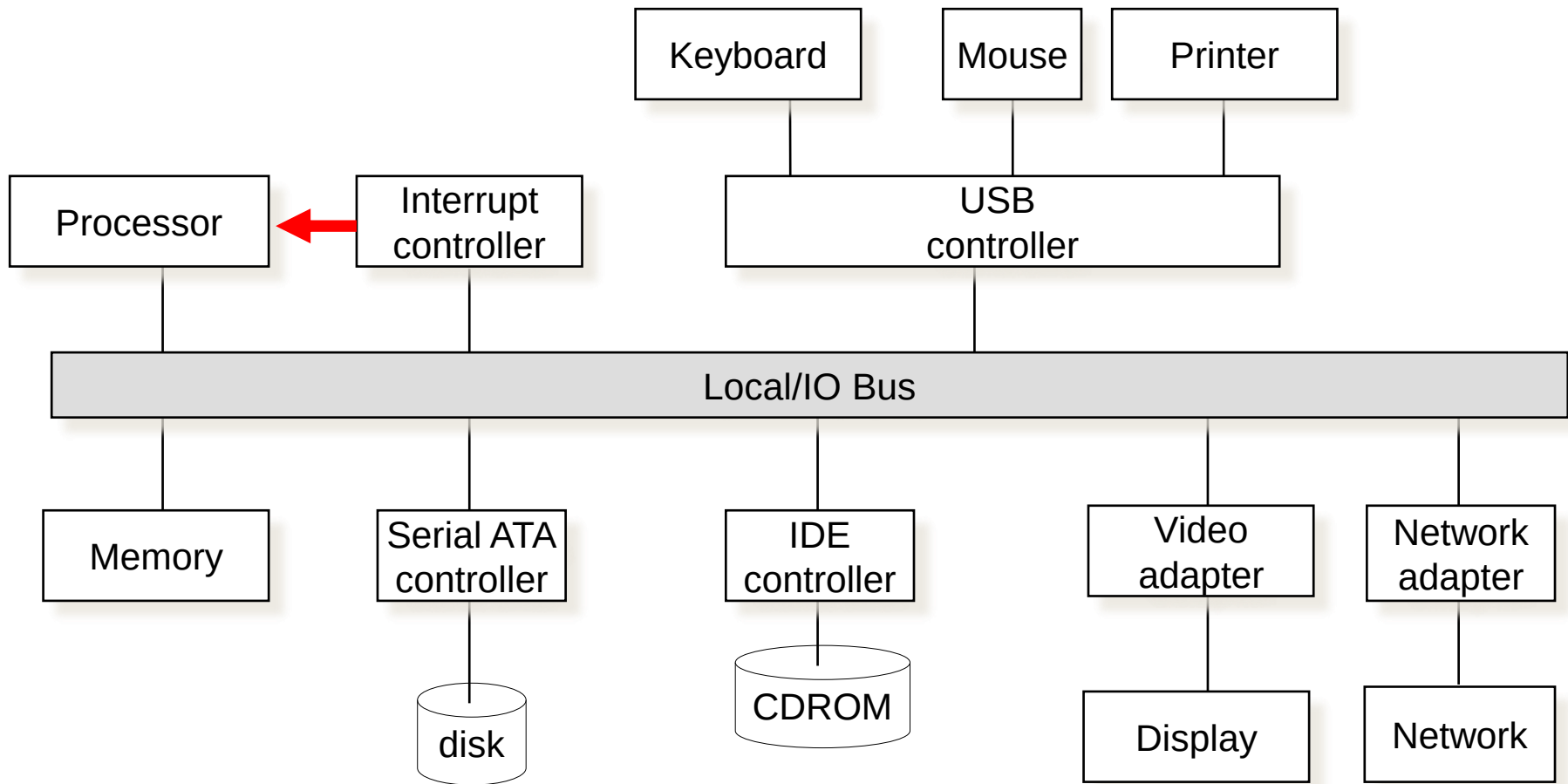
- **陷阱**：是执行一条指令的结果，在用户程序和内核之间提供系统调用接口。陷阱总返回到当前指令的下一条指令。
- **故障**：由错误引起，它可能会被故障处理程序修正，如果修正成功，将返回到当前正在执行的指令，重新执行。否则，处理程序返回到内核的abort例程，终止故障程序。故障的一个典型是缺页异常。
- **终止**：由不可恢复的错误造成的结果，处理程序将返回到内核中的abort例程，终止应用程序。
- **中断（异步中断）**：通常来自处理器外部IO设备的信号。例如网络适配器、磁盘控制器通过向处理器芯片上的一个管脚发信号，并将中断号（标识引发中断的设备）放在系统总线上，来触发中断。中断处理程序结束后返回到当前指令的下一条指令。

异常分类

■ 中断、陷阱、故障、终止



外部中断信号





异常处理

- RISC-V、MIPS
 - 设置一个异常原因寄存器（如MIPS中的Cause寄存器），用于记录异常原因。
 - 操作系统使用一个统一的异常处理程序，该程序按优先级顺序查询异常状态寄存器，识别出异常事件。
 - 例如：MIPS中位于内核地址0x8000 0180处有一个专门的异常处理程序。该异常处理程序会检测Cause寄存器以查明异常的具体原因，然后转到内核中相应的异常处理程序段中进行具体的处理。
- x86采用
 - 每个异常和中断都有一个中断号，根据此号，到中断描述符表中读取对应的具体的中断服务程序的入口地址。



MIPS 中的异常处理

- MIPS 对异常的处理流程
 - 保护现场，将各个寄存器的值压栈（处理完异常后恢复原来的指令流）。
 - 根据寄存器标志，判断是什么异常，并执行具体的异常处理函数。
 - ① 将出错指令的地址保存到异常程序计数器EPC（Exception Program Counter）
 - ② 设置CP0的Cause、STATUS寄存器
 - 32位Cause寄存器，其中5位记录异常原因（ExcCode）
 - 设置CP0中STATUS寄存器 EXL位为1，表示正在异常状态，这个时候CPU不能再响应中断，进入核心态。
 - 设置CP0中CAUSE寄存器（ExcCode位置为0，表示是中断异常），CAUSE寄存器 IP 位被设置为不同的中断号；有的异常也需设置BadVaddr寄存器。
 - ③ 将控制权交给操作系统，转到特定地址，根据异常原因执行异常处理程序
 - MIPS一般是0x8000 0180
 - 恢复现场，将栈里保存的寄存器内容写回。
 - 跳转到正常指令流的断点，继续执行。

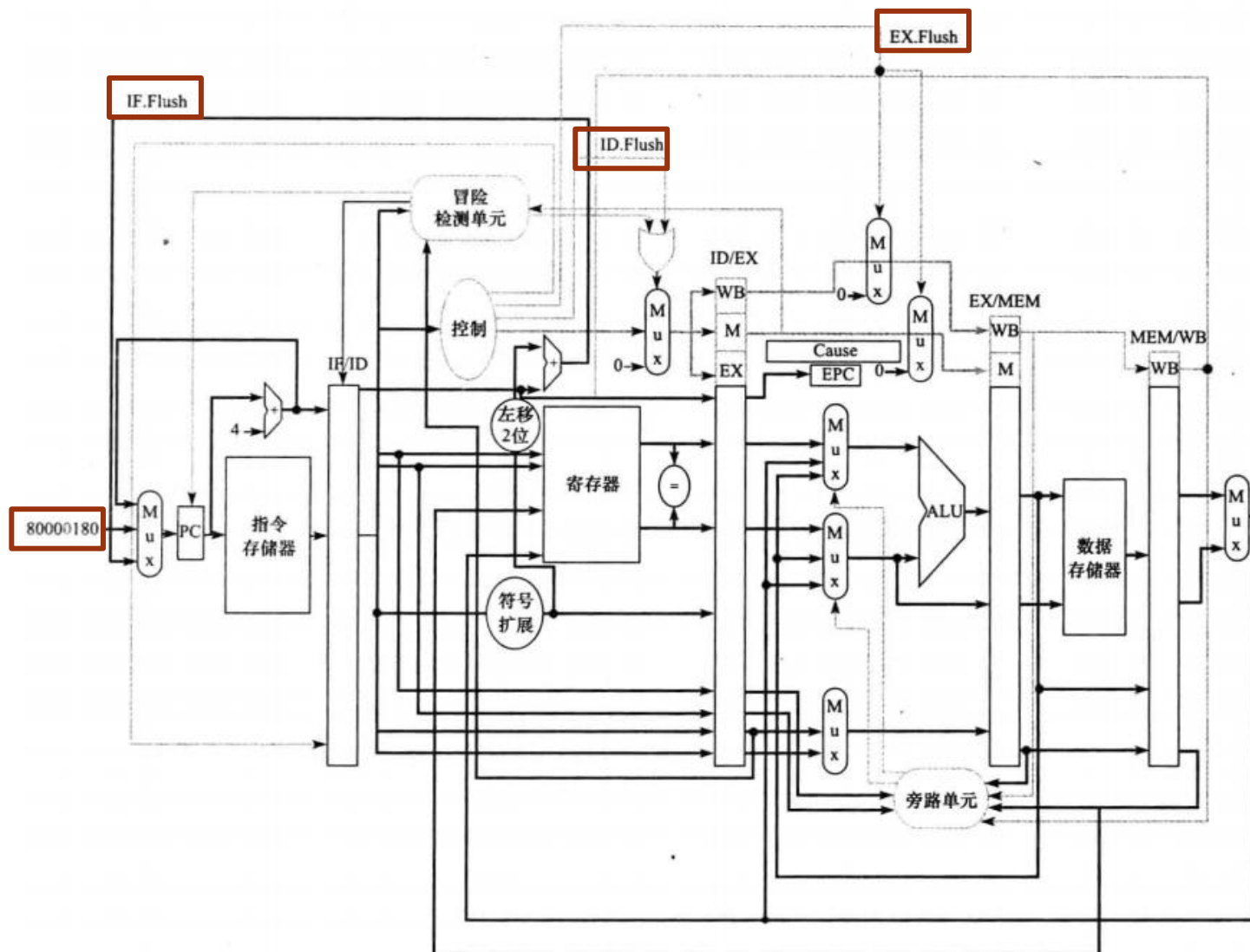


MIPS 流水线中异常处理

- 在多周期流水线CPU中支持异常和中断
 - 异常可以视为另一种形式的冒险
 - 出现异常，当前指令和流水线中的后续指令都暂时停止，跳转到异常处理程序中
 - IF：转换为nop指令
 - ID：多路选择器中增加控制器清零信号ID.Flush
 - EX：增加控制器清零信号EX.Flush

(暂时只考虑EX运算过程中出错类型的异常，或int/syscall等软中断)

MIPS 流水线中异常处理

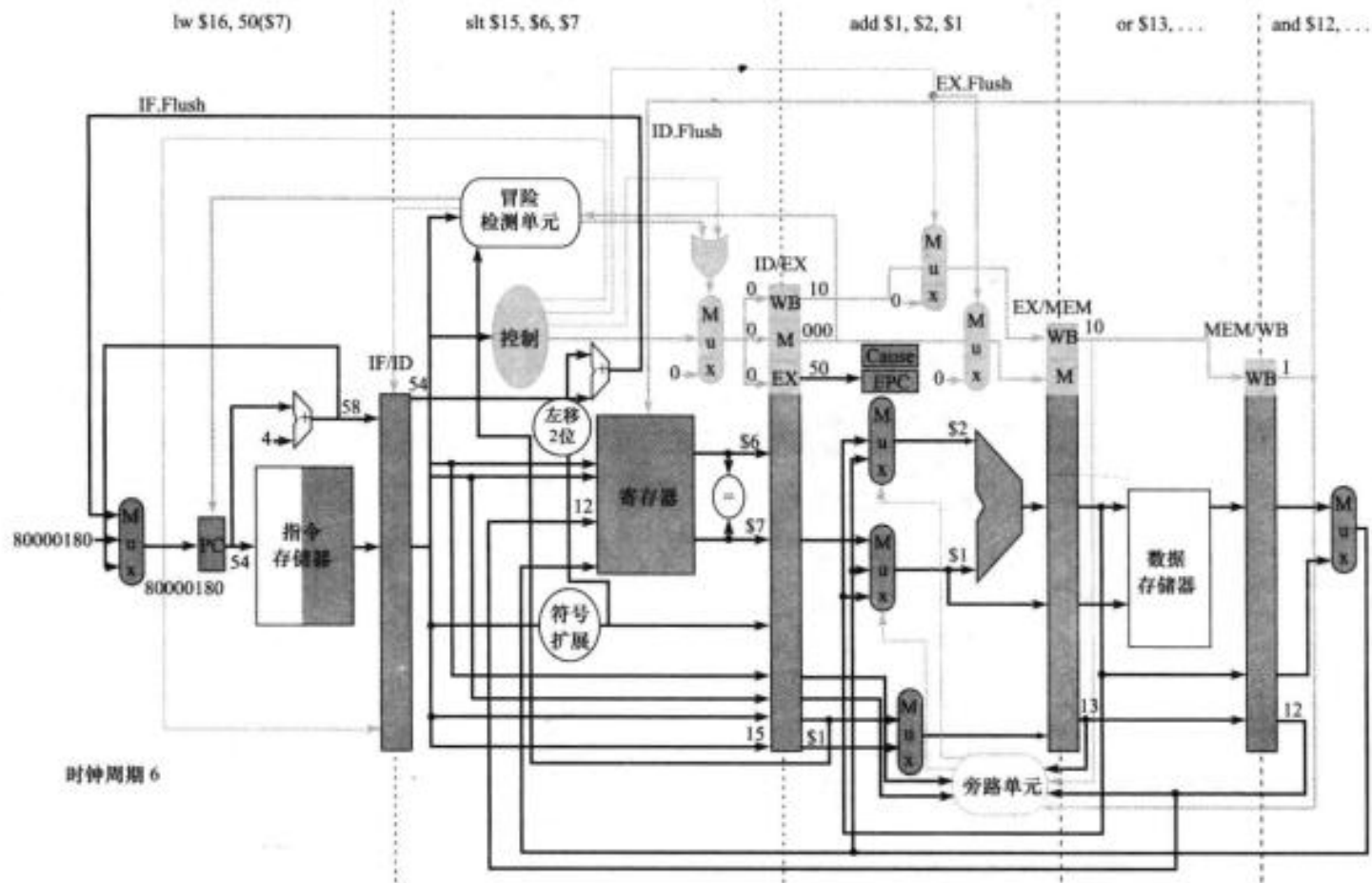




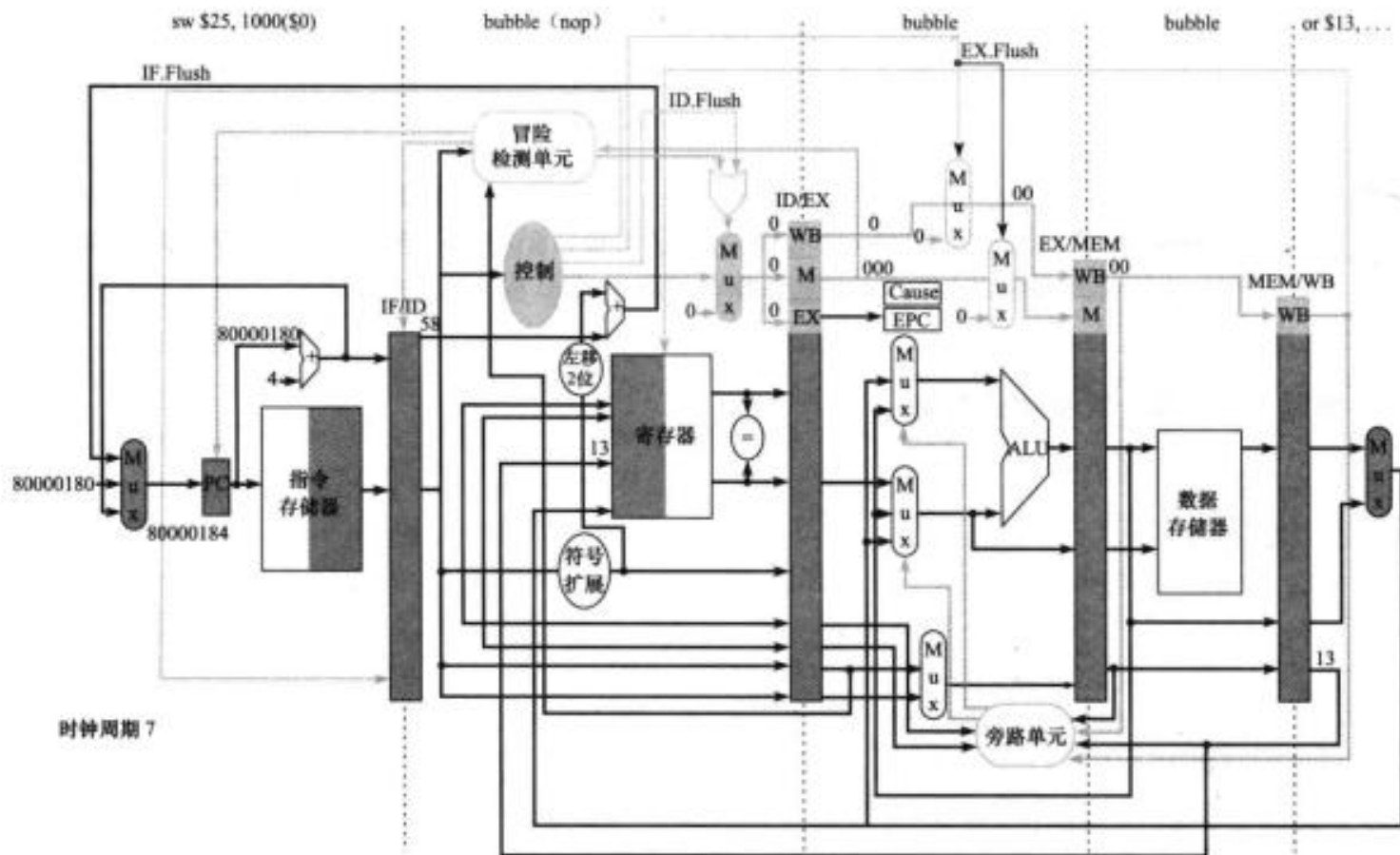
MIPS 流水线中异常处理

- [例子]给出以下指令序列:
 - 0x40 sub \$11, \$2, \$4
 - 0x44 and \$12, \$2, \$5
 - 0x48 or \$13, \$2, \$6
 - 0x4C **add** **\$1, \$2, \$1**
 - 0x50 slt \$15, \$6, \$7
 - 0x54 lw \$16, 50(\$7)
- 假定异常处理程序的开始部分如下:
 - 0x80000180 sw \$25, 1000(\$0)
 - 0x80000184 sw \$26, 1004(\$0)
- add指令发生溢出异常时流水线的情况

MIPS 流水线中异常处理



MIPS 流水线中异常处理





MIPS 流水线中异常处理

- 一般保存PC+4到EPC，中断处理程序再做-4操作
 - 非精确中断/异常
 - PC->EPC时，直接保存当时PC
 - 上述例子中，可能保存0x58
 - 硬件不精确，让操作系统去判定到底哪条指令发生异常，再通过跳转修正
- MIPS和主流CPU都支持精确中断



MIPS 异常处理

- MIPS中的协处理器 (CP)
 - CP0: 系统控制协处理器
 - CP1: 浮点数协处理器

CP0 寄存器

■ CP0中的寄存器

寄存器助记符	CP0寄存器编号	描述
SR	12	状态寄存器(Status Register)，一反常规的基本上由可写的控制位域组成。包括确定 CPU 特权等级、哪些中断引脚使能和其它的CPU 模式等位域。
Cause	13	什么原因导致异常或者中断？
EPC	14	异常程序计数器(Exception Program Counter)：异常/中断结束后从哪里重新开始执行。
Count	9	这两个寄存器一起形成了一个简单而有用的高分辨率定时器，频率（通常）为 CPU 流水线频率的一半。
Compare	11	
BadVaddr	8	导致最近的地址相关异常的程序地址。即使没有 MMU，各种地址错误也都会设置它。

CP0 寄存器

■ CP0中的寄存器

Conext	4	对存储器管理和地址转换硬件(TLB)编程的寄存器, 这在第 6 章讲到。
EntryHi	10	
EntryLo0-1	2-3	
Index	0	
PageMask	5	
Random	1	
Wired	6	
PRId	15	CPU 类型和版本号。类型号由 MIPS 公司管理, (至少) 当 CP0 寄存器集变动时应当不同。表 3.3 中列出了到 2004 年中为止用过的值。
Config	16	CPU 参数设置, 通常由系统决定; 一些可写, 另一些只读。有些 CPU 有高编号的专用寄存器。
Config1-3	16.1-3	
Ebase	15.1	异常入口点基址和——多 CPU 系统的——CPU ID。



CP0 寄存器

- 对CP0的访问

- `mtc0 rt, rd, <n>` #把通用寄存器rt数据传送到CP0的rd寄存器

- 例如 `mtc0 s, $12`: 把s送到CP0中的寄存器12

- `mfc0 rt, rd, <n>` #从CP0的rd寄存器中读出数据放到通用寄存器rt

- 例如：控制状态寄存器SR中特定的bit

```
mfc0    t0, SR
```

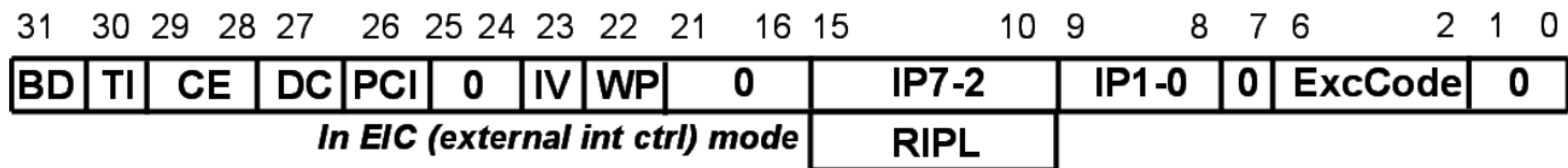
```
and     t0, <要清零的位的反码>
```

```
or      t0, <要置 1 的位>
```

```
mtc0    t0, SR
```

CP0 寄存器

- Cause寄存器
 - ExcCode (2-6) : 标识引起异常的原因
 - 8个独立的中断位
 - 其中6个为外部中断 (Cause.IP 7-2)
 - 2个为内部中断 (Cause.IP1-0) , 可由软件访问





异常类型

- 0: Interrupt,中断;
- 1: TLB Modified,试图修改 TLB 中映射为只读的内存地址;
- 2: TLB Miss Load,试图读取一个没有在 TLB 中映射到物理地址的虚拟地址;
- 3: TLB Miss Store,试图向一个没有在 TLB 中映射到物理地址的虚拟地址存入数据;
- 4: Address Error Load,试图从一个非对齐的地址读取信息;
- 5: Address Error Store,试图向一个非对齐的地址写入信息;
- 6: Instruction Bus Error,一般是指令 Cache 出错;
- 7: Data Bus Error,一般是数据 Cache 出错;
- 8: Syscall,由 syscall 指令产生。操作系统下,通用的由用户态进入内核态的方法。
- 9: Break Point,由 break 指令产生。最常见的 bp 指令,是由编译器产生的,在除法运算时插入一个 break point 指令,以达到在除 0 时抛出错误信息的目的。因此,如果在定位问题时发现了一个 Break Point 异常,且它的异常分代码为 07,应当考虑是出现了除 0 的情形;
- 10: RI,保留指令。在 CPU 执行到一条没有定义的指令时,进入此异常;



异常类型

- 11: Co-processor Unavailable,协处理器不可用。这个异常是由于试图对不存在的协处理器进行操作引起的。特别的,在没有浮点协处理器的处理器上执行这条命令,会导致这个异常。随之,操作系统会调用模拟浮点的 lib 库,来实现软件的浮点运算;
- 12: Overflow,算术溢出。只有带符号的运算会引起这个异常;
- 13: Trap,这个异常来源于 trap 指令。和 syscall 指令类似地,trap 指令也会引起一个异常,但 trap 指令可以附带一些条件,这样可以用于调试程序用。
- 14: VCEI,指令高速缓存中的虚地址一致性错误。(没明白怎么回事,还有待高手补充)
- 15: Float Point Exception,浮点异常;
- 16: Co-processor 2 Exception,协处理器 2 的异常;
- 17~22,留作将来的扩展;
- 23: Watch,内存断点异常。当设定了 WatchLo/WatchHi 两个寄存器时起作用。当 load/store 的虚拟地址 WatchLo/WatchHi 中匹配时,会引发这样一个异常;
- 24~30,留作将来的扩展



处理器

- 4.1 CPU概述
- 4.2 CPU设计的基本问题
- 4.3 MIPS模型机
- 4.4 单周期处理器设计
- 4.5 流水线技术
- 4.6 多周期处理器设计
- 4.7 流水线冒险
- 4.8 异常处理
- 4.9 指令间的并行
- 4.10 流水线结构实例



4.9 指令间的并行

- 并行处理
 - 指令级并行
 - 单核内部并行，提升整体性能（降低CPI）
 - 对上透明（隐藏）：非常受软件欢迎
 - 线程级并行
 - 数据级并行：SIMD, MIMD

指令间的并行

- 提高指令间并行的方法：更多的指令重叠执行
 - **提高流水线深度**：时钟周期变短，频率更高，性能更高
 - 理想情况下可以提高并发度
 - 受制于功耗，一般不会过深
 - **多发射**：每个流水级启动多条指令

处理器	年份	时钟频率	流水线级数	发射宽度	乱序/推测	核数目/片	功耗	
Intel 486	1989	25MHz	5	1	No	1	5	W
Intel Pentium	1993	66MHz	5	2	No	1	10	W
Intel Pentium Pro	1997	200MHz	10	3	Yes	1	29	W
Intel Pentium 4 Willamette	2001	2 000MHz	22	3	Yes	1	75	W
Intel Pentium 4 Prescott	2004	3 600MHz	31	3	Yes	1	103	W
Intel Core	2006	2 930MHz	14	4	Yes	2	75	W
Intel Core i5 Nehalem	2010	3 300MHz	14	4	Yes	1	87	W
Intel Core i5 Ivy Bridge	2012	3 400MHz	14	4	Yes	8	77	W

多发射

- 多发射 (multiple issue)
 - 需要增加一些硬件
 - 每个流水级可以同时处理多条指令 (没有冲突的情况下)
 - 可能让CPI小于1
 - 单发射流水线, CPI理想值是1
 - 4GHz四路多发射处理器, 每秒160亿条指令, $CPI=0.25$, $IPC=4$, 同时可能有20条指令在CPU中执行
 - 一般高端处理器每个时钟周期发射4-8条指令, 中端处理器发射2-3条



多发射

- 两种方法
 - 静态多发射 (static multiple issue) : 编译时, 软件决策
 - 动态多发射 (dynamic multiple issue) : 执行时, 硬件决策
- 需要处理两个主要任务
 - 打包指令放入发射槽: 决定发射多少条, 哪些指令
 - 处理数据冒险、控制冒险

静态多发射

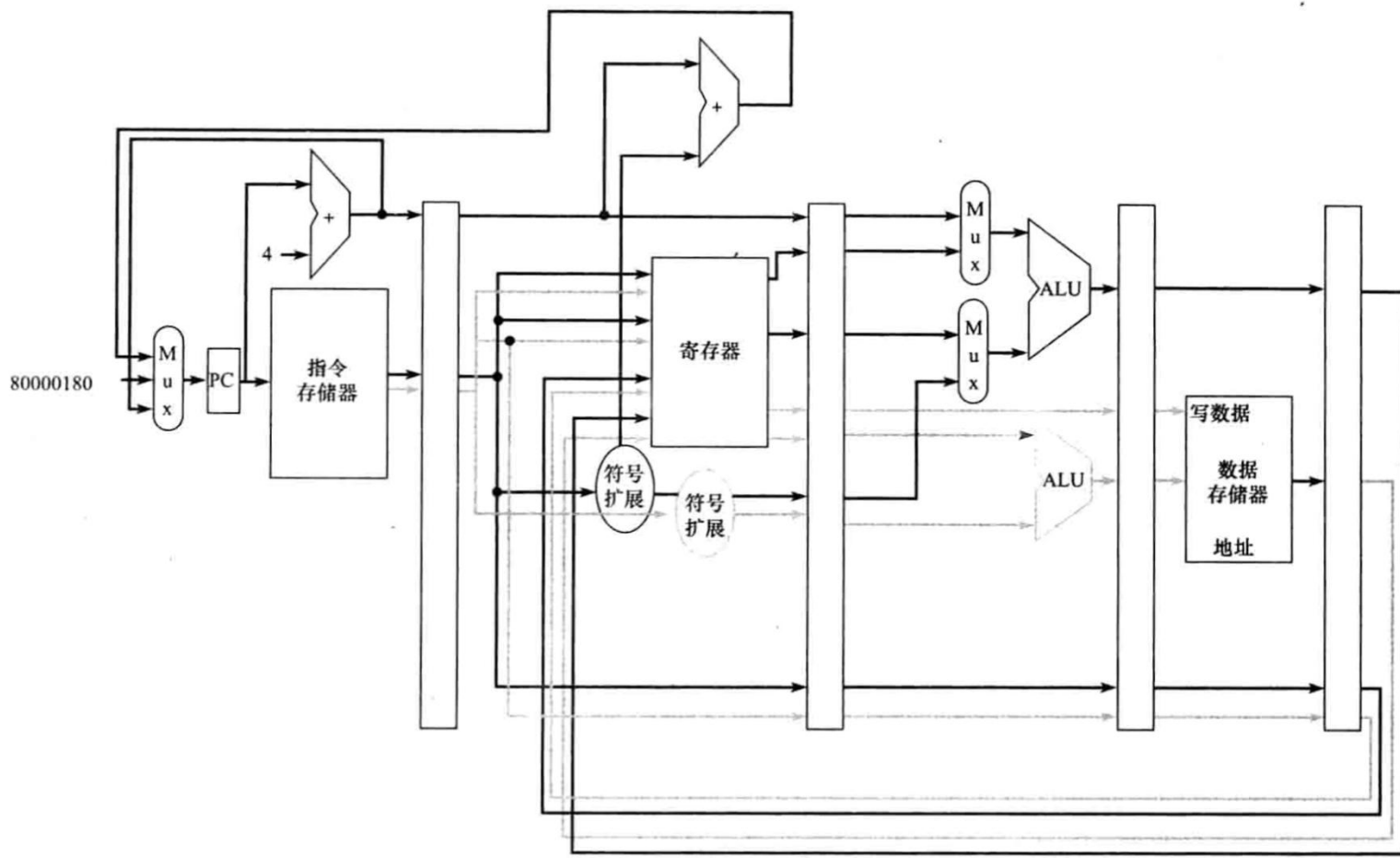
- 编译器支持指令打包和处理指令冒险
 - **发射包** (issue packet) : 一个时钟周期内同时发射的多条指令
 - 可被看作包含多个指令的长指令, 即**超长指令字** (Very Long Instruction Word, VLIW) : 一类可以同时启动多个操作的指令集
 - 同一周期可以同时发射的指令通常会有限制
 - 编译器处理好数据冒险和控制冒险
 - 静态分支预测和代码调度

静态多发射

- 例如：双发射（ALU或分支型+访存型）
 - If阶段一次取两条指令；没有合适的，编译器会加入nop指令凑数
 - Id阶段的寄存器要做硬件上的修改，解决结构冒险
 - 为ALU的输入读2个寄存器
 - 为存储操作计算地址读2个寄存器
 - ALU和Lw指令各自包含一个写端口
 - Ex部分会有多种运算硬件（额外的ALU用于地址计算）

指令类型	流水线阶段							
ALU 或分支	IF	ID	EX	MEM	WB			
load 或 store	IF	ID	EX	MEM	WB			
ALU 或分支		IF	ID	EX	MEM	WB		
load 或 store		IF	ID	EX	MEM	WB		
ALU 或分支			IF	ID	EX	MEM	WB	
load 或 store			IF	ID	EX	MEM	WB	
ALU 或分支				IF	ID	EX	MEM	WB
load 或 store				IF	ID	EX	MEM	WB

静态多发射数据通路



静态多发射示例

在一个 MIPS 静态双发射流水线中，下面这个循环将如何调度？

```
Loop: lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2# add scalar in $s2
      sw    $t0, 0($s1)# store result
      addi  $s1,$s1,-4# decrement pointer
      bne   $s1,$zero,Loop# branch $s1!=0
```

对该指令序列进行重排序，以尽可能地避免流水线阻塞。假设分支是可预测的，即控制冒险由硬件处理。

	ALU 或分支指令	数据传输指令	时钟周期
Loop:		lw \$t0,0(\$s1)	1
	addi \$s1,\$s1,-4		2'
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0,4(\$s1)	4

静态多发射示例

\$s1-16

- Lw指令若存在数据冒险，需阻塞
- 编译循环展开，更容易避免相关性
 - （上述例子中4个循环展开）
 - 注意\$s1值从第2周期开始已经减少16

0(\$s1)		16(\$s1)
-4(\$s1)		12(\$s1)
-8(\$s1)		8(\$s1)
-12(\$s1)		4(\$s1)

	ALU或分支指令	数据传输指令	时钟周期
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1, 12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1, 12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

静态多发射

- 反相关(antidependence), 即名字相关(name dependence)是因为重用寄存器名引起的相关, 不是真的数据相关 (WAW, WAR相关)
 - 循环展开后, 会有多对lw, sw, addu都用到\$t0, 但实际彼此没关系
 - 可以替换为不同的寄存器名 (\$t1, \$t2, ...) , 即寄存器重命名

	ALU 或分支指令	数据传输指令	时钟周期
Loop:		lw \$t0,0(\$s1)	1
	addi \$s1,\$s1,-4		2'
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0,4(\$s1)	4

动态多发射

- 动态多发射处理器：也称超标量（superscalar）处理器
 - 不管编译器怎么处理，由硬件进行调度，并保证正确性
 - 为什么需要动态多发射？
 - 静态多发射，即使sub准备好了，也不能发射，必须等lw和addu完成

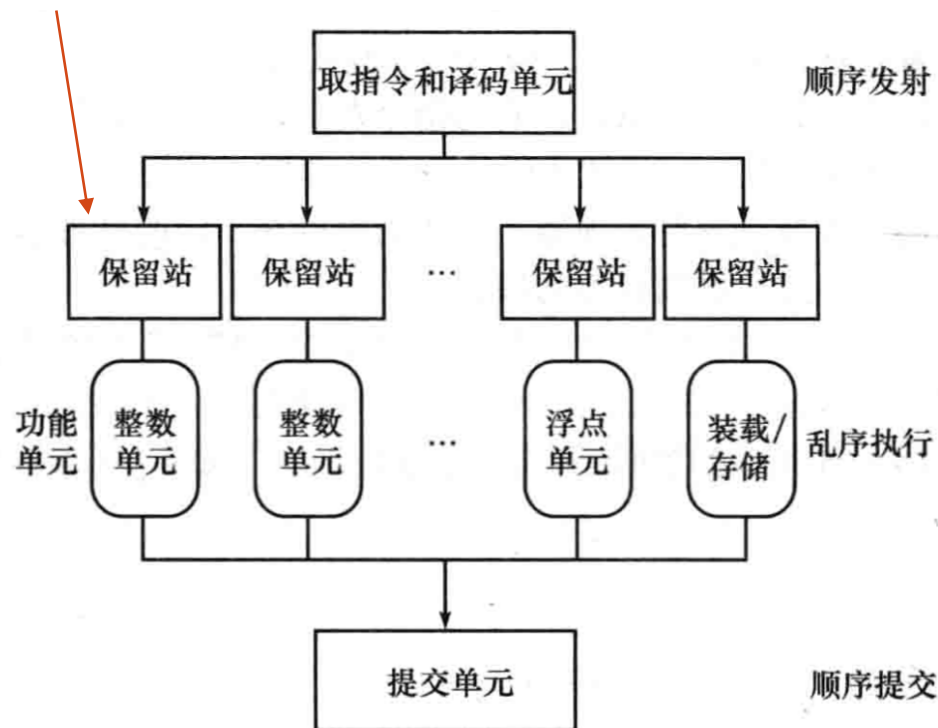
```
lw      $t0, 20($s2)
addu    $t1, $t0, $t2
sub      $s4, $s4, $t3
slti    $t5, $s4, 20
```

动态多发射

- 动态流水线调度
 - 由硬件选择当前周期执行的指令，尽量避免流水线的冒险和停顿
 - 流水线分为三个部分：
 - 取指和发射单元
 - 多功能部件：包含保留站和功能单元
 - 保留站：功能单元的缓冲区，保存指令的操作和所需操作数，只要操作数准备好，功能单元就绪，就计算结果
 - 提交单元：保存已完成指令的执行结果，包含重排序缓冲区，决定何时安全将操作结果送至reg/mem
 - 顺序发射（记录依赖关系）、乱序执行（可以不按程序指令序列执行），顺序提交

动态多发射

操作数如果在寄存器堆或重排序
缓冲区可用，就复制到保留站
(实现了寄存器重命名)



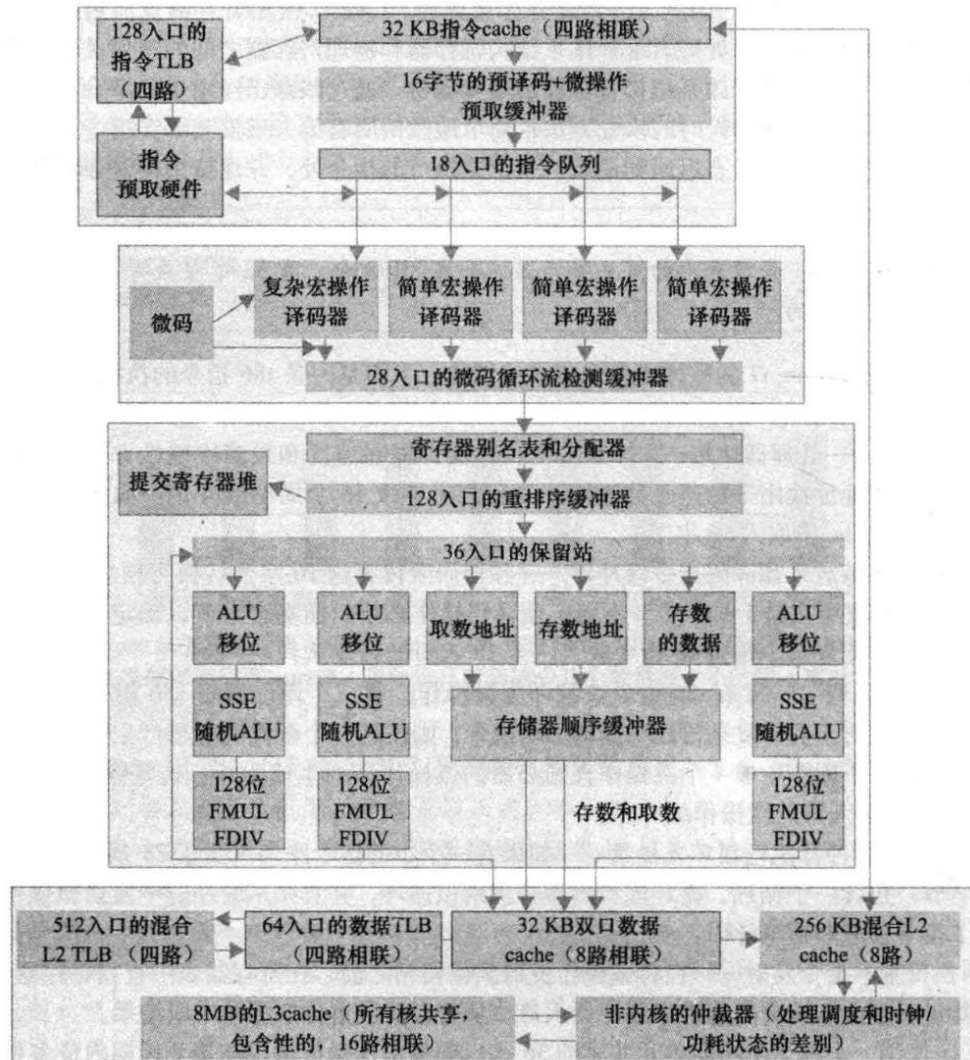


处理器

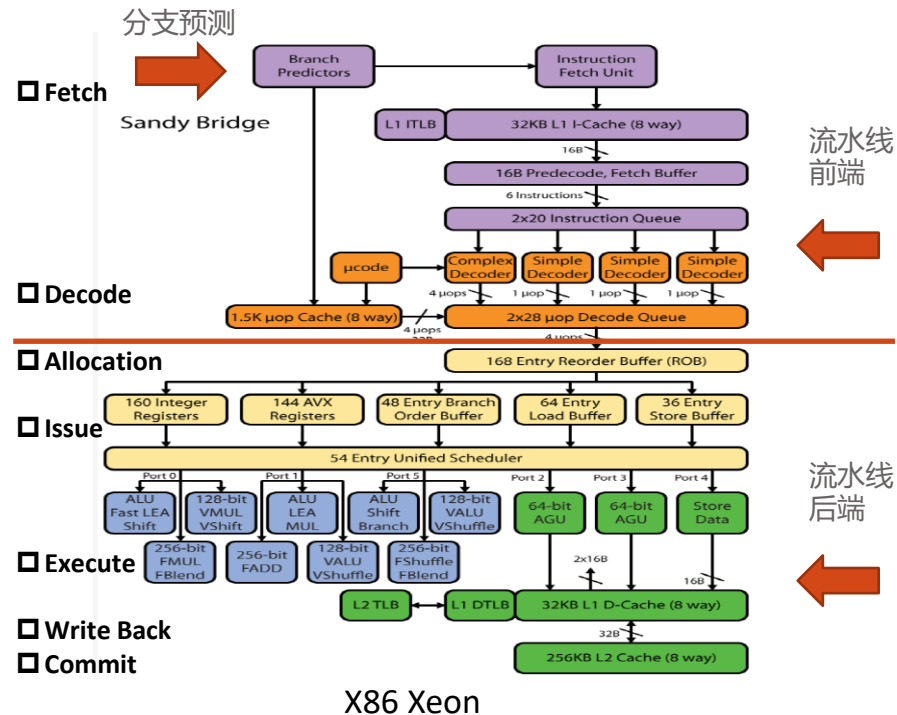
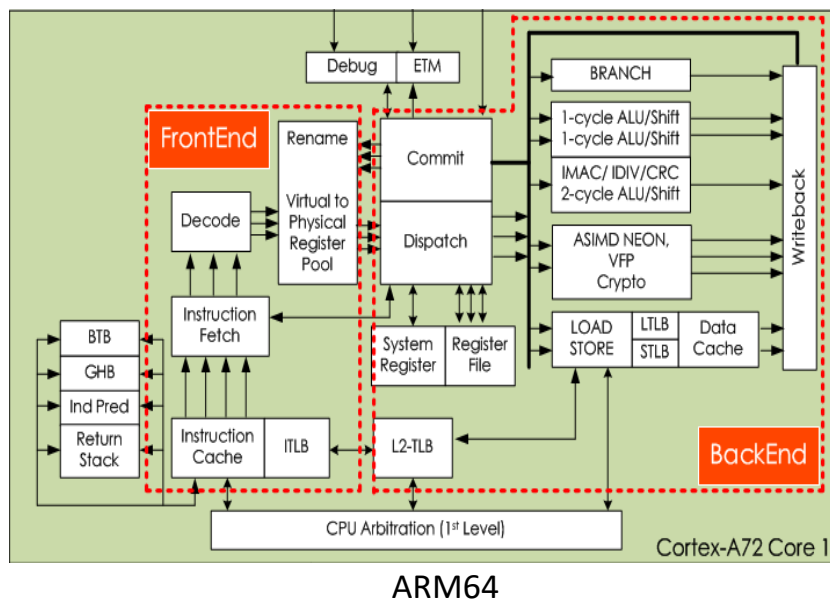
- 4.1 CPU概述
- 4.2 CPU设计的基本问题
- 4.3 MIPS模型机
- 4.4 单周期处理器设计
- 4.5 流水线技术
- 4.6 多周期处理器设计
- 4.7 流水线冒险
- 4.8 异常处理
- 4.9 指令间的并行
- 4.10 流水线结构实例

多发射

- Intel Core i7 920
 - 14级流水，动态多发射
 - X86指令 $\rightarrow$ 微操作（类MIPS指令）
 - 理想CPI=0.25, 但实际平均0.79



CPU流水线: ARM VS X86 XEON



ARM流水线与Xeon类似，按流水线前后端分解